T1 Programação Concorrente: Jantar dos Filósofos e Produtor/Consumidor em Go

Nomes: Pedro Widholzer e Felipe Flores

1. Primitivos de sincronização implementados

Não usamos `sync`, `sync/atomic` nem `context`, o que é verificado por `scripts/check.ps1`. Toda a sincronização é feita com channels.

- **`Mutex`:** `chan struct{}` de capacidade 1. *Invariante:* cheio significa travado e vazio significa livre. `Lock` envia e bloqueia se estiver cheio. `Unlock` recebe com `select/default` e dá `panic` se o channel estava vazio. `LockTimeout` faz `select` com `time.After`.
- **`Semaforo`:** `chan struct{}` de capacidade `max`, em que cada elemento é uma permissão. *Invariante:* `0 ≤ len(ch) ≤ max`. `Acquire` recebe. `Release` envia com `select/default` e dá `panic` se excederia `max`. Também existem `AcquireTimeout` e `Valor`, que devolve `len(ch)`.
- **`Latch`, no lugar do `WaitGroup`:** `chan struct{}` com buffer `n`. *Invariante:* `Done` nunca bloqueia e `Wait` recebe exatamente `n` vezes, logo só retorna quando todos sinalizaram.

Os testes com `go test -race` mostram que 8 goroutines com 100 mil incrementos cada, protegidas pelo `Mutex`, dão resultado exato. `Unlock` sem `Lock` e `Release` acima de `max` dão `panic`. O semáforo nunca admite mais de `max` simultâneos e `Wait` só retorna após `n` chamadas de `Done`.

2. Problema 1: Jantar dos Filósofos

2.1 Solução e decisões de projeto

Cada garfo é um `Mutex`. O filósofo `i` usa o garfo `i` à esquerda e o garfo `(i+1)%N` à direita. O laço do filósofo é único e executa exatamente **R ciclos** de pensar, ficar com fome, pegar os garfos, comer e soltar. Só a interface `Estrategia{Pegar, Soltar}` muda:

- **base:** pega o esquerdo, dorme `-pausa` (50 ms) e pega o direito.
- **hierarquia:** pega primeiro o garfo de menor índice.
- **limite:** um `Semaforo` com N−1 permissões, os lugares à mesa, é adquirido antes de pegar os garfos.
- **garçom:** uma goroutine é a única dona do estado dos garfos. Os filósofos pedem por um channel, e cada pedido traz o channel de resposta do filósofo. A liberação é avisada por outro channel. O garçom mantém uma **fila FIFO estrita** e só atende a cabeça da fila quando os dois garfos estão livres. Ninguém fura a fila, então não há starvation, ao custo de paralelismo.

Monitor e watchdog sem `sync`: os filósofos enviam eventos (`pensando`, `fome`, `pegou-esquerdo`, `comendo`, `terminou`) à **main**, única dona do estado. A main faz `select` entre receber um evento e `time.After(timeout)`. Se o tempo estoura com filósofos ativos, imprime `DEADLOCK DETECTADO` com o estado de cada um e termina com código 0, que é o resultado esperado da `base`. As

métricas ficam numa struct local de cada filósofo e seguem para a main no evento `terminou`, sem estado compartilhado.

2.2 Deadlock e starvation: condições de Coffman

Estratégia	Exclusão mútua	Posse e espera	Não preempção	Espera circular	Resultado
base	vale	vale	vale	**vale**	deadlock
hierarquia	vale	vale	vale	**quebrada**, pois os garfos são numerados	sem deadlock
limite	vale	vale	vale	**quebrada**, pois com no máximo N−1 sentados algum pega os 2 garfos	sem deadlock
garçom	vale	**quebrada**, pois recebe os 2 juntos ou nenhum	vale	não se forma	sem deadlock

Evidência da `base`: em `relatorio/dados/base-deadlock.txt`, com N=5 e R=100, o watchdog disparou após 2 s sem eventos. Os 5 filósofos seguravam o garfo esquerdo esperando o direito, com **0 refeições concluídas** cada.

Starvation: só o garçom o impede por construção, com a fila FIFO estrita. Em `hierarquia` e `limite`, os filósofos bloqueados num `Lock` ou `Acquire` esperam num channel, e o runtime do Go mantém essas filas em ordem de chegada, o que na prática evita starvation. Porém a especificação da linguagem não garante essa ordem, então não a tratamos como garantia. Nos experimentos, os 225 registros, um por filósofo por execução, completaram as R=100 refeições. Como R é fixo por filósofo, **a fairness é medida pelo tempo de espera**, da fome até os dois garfos, e não pelo número de refeições.

2.3 Fairness e custo

R=100 e pensar=comer=10 ms. Cada célula é a mediana [mín–máx] de 5 repetições.

estratégia	N	duração (ms)	espera média (ms)	espera máx (ms)	desvio das médias (ms)	Jain
hierarquia	5	2924 [2851–2942]	6.71 [6.52–6.96]	38.62 [29.11–49.14]	1.182 [0.912–1.514]	0.9682 [0.9527–0.9831]
limite	5	3196 [2842–4488]	10.85 [7.25–23.38]	43.69 [26.57–50.70]	0.446 [0.242–0.484]	0.9980 [0.9959–0.9999]
garçom	5	3434 [3226–4663]	12.46 [10.73–24.57]	44.14 [27.59–51.34]	0.439 [0.291–0.470]	0.9993 [0.9981–0.9998]
hierarquia	10	2942 [2941–3036]	6.79 [6.55–7.05]	38.75 [36.45–40.21]	1.346 [1.062–1.519]	0.9629 [0.9541–0.9757]
limite	10	3844 [3517–4394]	16.42 [13.53–22.17]	80.04 [52.65–85.88]	0.504 [0.436–0.567]	0.9991 [0.9985–0.9994]
garçom	10	4298 [3867–4715]	20.12 [16.98–23.43]	49.65 [47.01–81.03]	0.447 [0.300–0.614]	0.9996 [0.9990–0.9998]

- A **hierarquia é a mais rápida e tem a menor espera média, mas é a menos justa**, com Jain entre 0,96 e 0,97 e desvio das médias de 2,6 a 3 vezes o das outras duas. Para N=5, a espera total média por filósofo foi 813 ms no filósofo 0, 666, 592, 516 no filósofo 3 e 774 ms no filósofo 4. A assimetria vem do algoritmo: o filósofo 4, que usa os garfos 4 e 0, e o filósofo 0 pegam ambos o garfo 0 como primeira escolha, enquanto cada um dos demais disputa como primeira escolha apenas o garfo do próprio índice. Verificamos esse perfil apenas para N=5.
- **Limite e garçom são praticamente igualitários**, com Jain ≥ 0,998, mas custam tempo. Com N=10, a duração sobe de 2942 ms na hierarquia para 3844 ms no limite

(+31%) e 4298 ms no garçom (+46%). A espera média vai de 6,79 para 16,42 e 20,12 ms.

- O custo maior do garçom é coerente com seu desenho: as decisões passam por uma única goroutine e só a cabeça da fila é atendida, o que reduz o paralelismo. Não medimos cada efeito isoladamente.
- Em N=5 as faixas de `limite` e `garçom` são largas, com duração até 4488 e 4663 ms, e se sobrepõem. Com 5 repetições não distinguimos essas duas estratégias entre si em duração.

2.4 Encerramento ordenado

Cada filósofo sai do laço após R ciclos, solta os garfos, envia `terminou` e chama `Latch.Done`. A main aguarda o `Latch`. No caso do garçom, **a main fecha o channel de pedidos** e espera a goroutine do garçom sair pelo channel `fim`. Ao final imprime `runtime.NumGoroutine() = 1`. A exceção é a `base`, em que o watchdog detecta o deadlock e o processo termina com `os.Exit(0)`.

3. Problema 2: Produtor/Consumidor

3.1 Solução e decisões de projeto

Produtores e consumidores são idênticos nas duas versões e usam a interface `Buffer{Put, Get(timeout), Len, Encerrar}`.

- **channel:** o buffer é `make(chan Item, K)`. `Get` com timeout usa `select` com `time.After`. `Encerrar` é o `close(ch)`, feito pela main.
- **sem:** anel `[K]Item` com índices `in` e `out`, `notFull` começando em K e `notEmpty` começando em 0. `Put` faz `notFull.Acquire`, `muIn.Lock`, escreve, `muIn.Unlock` e `notEmpty.Release`. `Get` é o espelho, com `muOut`. Os semáforos garantem que há vaga ou item. Os **dois mutexes** impedem dois produtores no mesmo slot e dois consumidores no mesmo item e, por serem separados, deixam um produtor e um consumidor trabalharem ao mesmo tempo. `Encerrar(c)` insere `c` poison pills, e como o anel é FIFO todos os itens reais saem antes.
- **Timeout:** só o consumidor 0 usa `Get(timeout)`. Ao estourar, conta um ocioso, imprime uma linha e volta ao laço, sem consumir nada. A demonstração está no README, com `-prod-delay=50ms -timeout=20ms`. Nos benchmarks a coluna `timeouts` é 0 em todas as execuções, pois o timeout padrão de 200 ms nunca estourou.
- **Verificação final:** o total consumido deve ser igual ao produzido, nenhum par `(ProdID, Seq)` pode ser repetido ou inesperado, conferido por um mapa na main, e `NumGoroutine` deve ser 1. Se falhar, `os.Exit(1)`.
- **Métricas:** throughput, itens por consumidor, ocupação média e timeouts. A ocupação é amostrada a cada 1 ms por uma goroutine parada pelo channel `quit` antes do encerramento.

3.2 Efeito de K e comparação channel e semáforos

Em todos os cenários P=4 e C=4. No cenário **A** os produtores são mais rápidos, com prod 1 ms, cons 3 ms e 300 itens por produtor. No **B** os consumidores são mais rápidos, com prod

3 ms, cons 1 ms e 300 itens por produtor. No **C** não há atraso e são 50 000 itens por produtor, para isolar o custo da sincronização. Throughput em itens/s, mediana [mín–máx] de 5 repetições:

cenário	K	channel	sem
A	1	1013 [812–1125]	1151 [1131–1170]
A	10	979 [782–1052]	1101 [1018–1174]
A	100	1040 [905–1061]	1132 [1110–1162]
B	1	1082 [978–1152]	986 [946–1024]
B	10	1116 [1102–1163]	1014 [995–1097]
B	100	1089 [967–1162]	1111 [1072–1135]
C	1	1110394 [999139–1115654]	707916 [596604–741088]
C	10	1878504 [1376471–2406255]	704655 [655139–710047]
C	100	3113548 [1937050–3573158]	756470 [729487–994225]

- **Ocupação média:** no cenário A o buffer fica praticamente cheio nas duas versões, com 0,97 a 0,98 para K=1, 9,9 para K=10 e 95,6 a 96,3 para K=100. No B fica praticamente vazio, perto de 0 em todos os K. Isso confirma qual lado é o lento em cada cenário.
- **Cenários A e B, dominados pelo `Sleep`:** o throughput fica entre 979 e 1151 itens/s, cerca de 73% a 86% do teto teórico do lado lento, que é 4 goroutines divididas por 3 ms, ou 1333 itens/s. Atribuímos a diferença ao jitter e ao excesso dos `Sleep`, o que é uma hipótese não medida. **K quase não altera o resultado.** Entre as versões, `sem` tem a mediana maior no A, de 9% a 14% acima do `channel`, e `channel` tem a maior no B para K=1 e 10, cerca de 10%. Como o sentido se inverte entre os cenários, tratamos isso como ruído de agendamento e de timers, e não como custo de sincronização.
- **Cenário C, só sincronização:** o `channel` escala com K, de 1,11 M para 1,88 M e 3,11 M itens/s, e o `sem` fica praticamente plano, de 708 k para 705 k e 756 k. Com K=100, o `channel` faz cerca de 4,1 vezes o throughput do `sem`. A explicação mais provável, não medida isoladamente, é que cada `Put` ou `Get` do `sem` custa 4 operações em channel (Acquire, Lock, Unlock e Release) contra 1 do `channel`, e que `muIn` e `muOut` serializam produtores entre si e consumidores entre si, algo que aumentar K não alivia.
- **Limitação de medição:** em execuções muito curtas a duração é limitada pela resolução do relógio do Windows, e um teste inicial com 1000 itens por produtor mediu duração 0. Por isso o cenário C usa 50 000 itens por produtor.

3.3 Deadlock e starvation

Não há espera circular. O produtor só espera por vaga, o consumidor só por item, e cada operação libera o mutex antes do `Release` do semáforo, então a espera termina assim que o outro lado progride. O algoritmo não impede formalmente que um produtor ou consumidor específico seja preterido. Nos dados de `itens_por_consumidor`, nos cenários A e B todo consumidor processou pelo menos 289 itens no `channel` e 292 no `sem`, de uma média de 300, ou seja, nenhum ficou parado. No cenário C a distribuição é bem mais desigual no `channel`: o consumidor 0 foi o que menos consumiu em 15 de 15 execuções, com média de 23 447 itens contra 58 851 dos demais. No `sem` isso ocorreu em 11 de 15, com 47 338 contra 50 887. A causa provável é que só o consumidor 0 cria um timer com `time.NewTimer` a cada `Get`,

o que o torna mais lento por item. O efeito é maior no `channel`, onde o resto da operação é mais barato. Não testamos isso isoladamente.

3.4 Encerramento ordenado

Os produtores chamam `Latch.Done` ao terminar. A main espera todos, para o amostrador pelo `quit` e só então chama `Encerrar(c)`. Quem encerra é **a main, nunca um produtor**. No `channel` ela fecha o channel, e os consumidores drenam o resto e recebem `Closed`. No `sem` ela insere `c` pills, e cada consumidor consome uma e sai. A main espera o `Latch` dos consumidores, recebe de cada um seu slice de itens por um channel com buffer e faz a verificação final.

4. Uso de IA

A arquitetura das soluções e o raciocínio de como resolver cada problema foram definidos e desenvolvidos pelo grupo, que os descreveu em um roteiro detalhado. Isso inclui as estratégias contra deadlock, o monitor e watchdog por channels, os primitivos sobre channels, o encerramento ordenado e o desenho dos experimentos. A ferramenta de IA, o Claude Code com o modelo Sonnet 5.5, foi usada apenas para **gerar o código** e os **testes** a partir desse roteiro, além de scripts de apoio em PowerShell e de ajustes de ambiente e medição, como a instalação de Go e gcc, o SSH, o PowerShell 5.1 e a duração 0 no Windows. Os resultados dos experimentos foram **conferidos pelo grupo**, que executou e revisou o código. O registro completo está em `AI-USO.md`.

5. Referências

- Dijkstra, E. W. Hierarchical ordering of sequential processes. *Acta Informatica*, 1971.
- Coffman, E. G.; Elphick, M. J.; Shoshani, A. System Deadlocks. *ACM Computing Surveys*, 3(2), 1971.
- Jain, R.; Chiu, D.; Hawe, W. *A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems*. DEC-TR-301, 1984.
- Tanenbaum, A. S.; Bos, H. *Modern Operating Systems*. Pearson.
- Documentação de Go: *The Go Memory Model* e os pacotes `time`, `flag` e `runtime`.
- Enunciado do T1, FPPD, PUCRS.